mindflux's Notebook

Abdullah Al Ahad Bhuiyan

Contents

1 Algorithms **2**
1.1 Matrix Exponential (Tribonacci) . . . 2
1.2 Mo's Algorithm . . . 2

2 Data Structures **3**
2.1 DSU . . . 3
2.2 Policy Based Data Structure . . . 3
2.3 Prefix Sum 2D . . . 3
2.4 Segment Tree Sum . . . 3

3 Geometry **4**
3.1 Angle Beetween Two Vector . . . 4
3.2 Area of a Polygon . . . 4
3.3 Distance from a Point to a Line . . . 4
3.4 Geometry Formula . . . 4
3.5 Pyramid & Frustum . . . 4
3.6 Intersection Point of Two Lines . . . 5
3.7 Point Position Relative to a Line . . . 5
3.8 Point . . . 5
3.9 Polar Angle . . . 5
3.10 Segment Intersection Check . . . 5

4 Graph **6**
4.1 Dijkstra . . . 6

5 Mathematics **6**
5.1 Combinatorics . . . 6
5.2 Fast Exponentiation . . . 6
5.3 Prime Generator . . . 6

6 Misc **6**
6.1 ahad . . . 6
6.2 Custom Comperator . . . 7

7 Strings **7**
7.1 KMP Algorithm . . . 7

1 Algorithms

1.1 Matrix Exponential (Tribonacci)

```
// Function to multiply two 3x3 matrices
void multiply(vector<vector<ll> >& A,
              vector<vector<ll> >& B)
{
    // Matrix to store the result
    vector<vector<ll> > C(3, vector<ll>(3));

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            for (int k = 0; k < 3; k++) {
                C[i][j]
                    = (C[i][j] + ((A[i][k]) * (B[k][j])));
            }
        }
    }

    // Copy the result back to the first matrix
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            A[i][j] = C[i][j];
        }
    }
}

// Function to calculate (Matrix M) ^ expo
vector<vector<ll> >
power(vector<vector<ll> > M, int expo)
{
    // Initialize result with identity matrix
    vector<vector<ll> > ans
        = { { 1, 0, 0 }, { 0, 1, 0 }, { 0, 0, 1 } };

    // Fast Exponentiation
    while (expo) {
        if (expo & 1)
            multiply(ans, M);
        multiply(M, M);
        expo >>= 1;
    }

    return ans;
}

// function to return the Nth tribonacci number
ll tribonacci(int n)
{
    // base condition
    if (n == 0 || n == 1)
        return n;

    // Matrix M to generate the next tribonacci number
    vector<vector<ll> > M
        = { { 1, 1, 1 }, { 1, 0, 0 }, { 0, 1, 0 } };

    // Since first 3 number of tribonacci series are:
    // trib(0) = 0
    // trib(1) = 1
    // trib(2) = 1
    // F = {{trib(2), 0, 0}, {trib(1), 0, 0}, {trib(0), 0,
    // 0}}
    vector<vector<ll> > F
        = { { 1, 0, 0 }, { 1, 0, 0 }, { 0, 0, 0 } };

    vector<vector<ll> > res = power(M, n - 2);
    multiply(res, F);

    return res[0][0];
}
```

1.2 Mo's Algorithm

```
// Variable to represent block size. This is made global
// so compare() of sort can use it.
int block;

// Structure to represent a query range
struct Query
{
    int L, R;
};

// Function used to sort all queries so that all queries
// of the same block are arranged together and within a
     block,
// queries are sorted in increasing order of R values.
bool compare(Query x, Query y)
{
    // Different blocks, sort by block.
    if (x.L/block != y.L/block)
        return x.L/block < y.L/block;

    // Same block, sort by R value
    return x.R < y.R;
}

// Prints sum of all query ranges. m is number of queries
// n is size of array a[].
void queryResults(int a[], int n, Query q[], int m)
{
    // Find block size
    block = (int)sqrt(n);

    // Sort all queries so that queries of same blocks
    // are arranged together.
    sort(q, q + m, compare);

    // Initialize current L, current R and current sum
    int currL = 0, currR = 0;
    int currSum = 0;

    // Traverse through all queries
    for (int i=0; i<m; i++)
    {
        // L and R values of current range
        int L = q[i].L, R = q[i].R;

        // Remove extra elements of previous range. For
        // example if previous range is [0, 3] and current
        // range is [2, 5], then a[0] and a[1] are subtracted
        while (currL < L)
        {
            currSum -= a[currL];
            currL++;
        }

        // Add Elements of current Range
        while (currL > L)
        {
            currSum += a[currL-1];
            currL--;
        }
        while (currR <= R)
        {
            currSum += a[currR];
            currR++;
        }

        // Remove elements of previous range. For example
        // when previous range is [0, 10] and current range
        // is [3, 8], then a[9] and a[10] are subtracted
        while (currR > R+1)
        {
            currSum -= a[currR-1];
            currR--;
```

```
        }

        // Print sum of current range
        cout << "Sum of [" << L << ", " << R
             << "] is " << currSum << endl;
    }
}

int a[] = {1, 1, 2, 1, 3, 4, 5, 2, 8};
int n = sizeof(a)/sizeof(a[0]);
Query q[] = {{0, 4}, {1, 3}, {2, 4}};
int m = sizeof(q)/sizeof(q[0]);
queryResults(a, n, q, m);
```

2 Data Structures

2.1 DSU

```
class UnionFind {
    vector<int> parent;
public:
    UnionFind(int size) {

        parent.resize(size);

        // Initialize the parent array with each
        // element as its own representative
        for (int i = 0; i < size; i++) {
            parent[i] = i;
        }
    }

    // Find the representative (root) of the
    // set that includes element i
    int find(int i) {

        // If i itself is root or representative
        if (parent[i] == i) {
            return i;
        }

        // Else recursively find the representative
        // of the parent
        return find(parent[i]);
    }

    // Unite (merge) the set that includes element
    // i and the set that includes element j
    void unite(int i, int j) {

        // Representative of set containing i
        int irep = find(i);

        // Representative of set containing j
        int jrep = find(j);

        // Make the representative of i's set
        // be the representative of j's set
        parent[irep] = jrep;
    }
};

UnionFind uf(size);
uf.unite(1, 2);
uf.unite(3, 4);
bool inSameSet = (uf.find(1) == uf.find(2));
cout << "Are 1 and 2 in the same set? "
       << (inSameSet ? "Yes" : "No") << endl;
```

2.2 Policy Based Data Structure

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<typename T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;
```

2.3 Prefix Sum 2D

```
vector<vector<ll>> prefixSum2D(vector<vector<ll>> &arr) {
    int n = arr.size();
    int m = arr[0].size();

    vector<vector<ll>> prefix(n, vector<ll>(m, 0));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            prefix[i][j] = arr[i][j];
            if (i > 0)
                prefix[i][j] += prefix[i - 1][j];

            if (j > 0)
                prefix[i][j] += prefix[i][j - 1];

            if (i > 0 && j > 0)
                prefix[i][j] -= prefix[i - 1][j - 1];

        }
    }

    return prefix;
}

ll query(auto& prefix, int r1, int c1, int r2, int c2) {

    ll total = prefix[r2][c2];
    ll top = (r1 > 0) ? prefix[r1 - 1][c2] : 0;
    ll left = (c1 > 0) ? prefix[r2][c1 - 1] : 0;
    ll overlap = (r1 > 0 && c1 > 0) ? prefix[r1 - 1][c1 - 1]
         : 0;

    int sum = total - top - left + overlap;
    return sum;
}
```

2.4 Segment Tree Sum

```
const int N = 2e5;

struct Segment {
    ll sum;
} seg_tree[2*N];

void recalculate(int node) {
    seg_tree[node].sum = seg_tree[(node << 1) + 1].sum +
         seg_tree[(node << 1) + 2].sum;
}

void build(vector<int> &a, int node, int left, int right) {
    if(left == right)
        seg_tree[node].sum = a[left];
    else {
        int mid = (left + right) >> 1;
        build(a, (node << 1) + 1, left, mid);
        build(a, (node << 1) + 2, mid+1, right);
        recalculate(node);
    }
}
```

```
void update(int node, int left, int right, int x, int y) {
    if(left == right)
        seg_tree[node].sum = y;
    else {
        int mid = (left + right) >> 1;
        if(x <= mid)
            update((node << 1) + 1, left, mid, x, y);
        else
            update((node << 1) + 2, mid+1, right, x, y);

        recalculate(node);
    }
}

ll query(int node, int left, int right, int x, int y) {
    if(x <= left && right <= y)
        return seg_tree[node].sum;

    int mid = (left + right) >> 1;
    ll ans = 0;
    if(x <= mid)
        ans += query((node << 1) + 1, left, mid, x, y);
    if(y >= mid + 1)
        ans += query((node << 1) + 2, mid + 1, right, x, y);

    return ans;
}
```

3 Geometry

3.1 Angle Beetween Two Vector

```
// Returns angle in radians [0, pi]
ld angleBetween(Point a, Point b) {
    ld d = dot(a, b) / (dis(a, Point(0,0)) * dis(b, Point
        (0,0)));
    d = max(-1.0ld, min(1.0ld, d)); // Clamp for numerical
        stability
    return acos(d);
}
```

3.2 Area of a Polygon

```
ld polygonArea(vector<Point> &poly) {
    ld area = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        Point curr = poly[i], nxt = poly[(i + 1) % n];
        area += cross(curr, nxt);
    }
    return abs(area) / 2.0;
}
```

3.3 Distance from a Point to a Line

```
ld distPointToLine(Point p, Point a, Point b) {
    // Area of triangle ABP = 0.5 * base * height -> height =
        2*Area / base
    return abs(cross(b - a, p - a)) / dis(a, b);
}

ld distPointToSegment(Point p, Point a, Point b) {
    Point ab = b - a, ap = p - a, ba = a - b, bp = p - b;
    if (dot(ab, ap) <= 0) return dis(p, a);
    if (dot(ba, bp) <= 0) return dis(p, b);
    return abs(cross(ab, ap)) / dis(a, b);
}
```

3.4 Geometry Formula

1. Distances

- Euclid: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- Manhat: $d = |x_2 - x_1| + |y_2 - y_1|$
- Line $(Ax + By + C = 0)$: $\frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$

2. Triangles

- Heron Area: $\sqrt{s(s-a)(s-b)(s-c)}$ where $s = (a+b+c)/2$
- Cosine: $c^2 = a^2 + b^2 - 2ab\cos(C)$
- Sine: $\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$
- $R = \frac{abc}{4A}$, $r = \frac{A}{s}$

3. Circle Sphere

- **Circle:** Area $= \pi r^2$, Circumference $= 2\pi r$
- **Sector Area:** $\frac{\theta}{360^\circ}\pi r^2$ $\quad$ (θ *in degrees*)
- **Arc Length:** $\frac{\theta}{360^\circ}2\pi r = r\theta$ $\quad$ (θ *in radians*)
- **Sphere:** Volume $= \frac{4}{3}\pi r^3$, Surface Area $= 4\pi r^2$
- **Spherical Cap (Height h):**

$$V = \frac{1}{3}\pi h^2(3r - h), \quad A_{\text{lateral}} = 2\pi r h$$

4. Cylinder Cone

- **Right Circular Cylinder:**

$$\text{Lateral Area} = 2\pi r h, \quad \text{Total Area} = 2\pi r(r + h)$$
$$\text{Volume} = \pi r^2 h$$

- **Hollow Cylinder (Pipe, inner r, outer R):**

$$\text{Volume} = \pi(R^2 - r^2)h$$

- **Right Circular Cone (Radius r, Height h, Slant $l = \sqrt{r^2 + h^2}$):**

$$\text{Lateral Area} = \pi r l, \quad \text{Total Area} = \pi r(r + l)$$
$$\text{Volume} = \frac{1}{3}\pi r^2 h$$

3.5 Pyramid & Frustum

- **General Pyramid (B = Base Area, h = Height):**

$$\text{Volume} = \frac{1}{3}Bh$$
$$\text{Total Area} = B + \frac{1}{2} \times (\text{Perimeter of Base}) \times l$$

- **Regular Square Pyramid (Base side a, Height h):**

$$V = \frac{1}{3}a^2 h, \quad l = \sqrt{h^2 + (a/2)^2}$$

- **Frustum of a Cone (Radii r_1, r_2, Height h):**

$$V = \frac{1}{3}\pi h(r_1^2 + r_2^2 + r_1 r_2)$$
$$A_{\text{lat}} = \pi(r_1 + r_2)\sqrt{h^2 + (r_1 - r_2)^2}$$

3.6 Intersection Point of Two Lines

```
Point lineIntersection(Point a, Point b, Point c, Point d) {
    ld a1 = b.y - a.y, b1 = a.x - b.x, c1 = a1 * a.x + b1 * a
        .y;
    ld a2 = d.y - c.y, b2 = c.x - d.x, c2 = a2 * c.x + b2 * c
        .y;
    ld determinant = a1 * b2 - a2 * b1;

    // Lines are parallel if determinant == 0
    return Point((c1 * b2 - c2 * b1) / determinant, (a1 * c2
        - a2 * c1) / determinant);
}
```

3.7 Point Position Relative to a Line

```
// > 0: Left side (Counter-Clockwise)
// < 0: Right side (Clockwise)
// = 0: Collinear
ld orientation(Point a, Point b, Point p) {
    return cross(b - a, p - a);
}
```

3.8 Point

```
struct Point{
    ld x, y;
    Point() {
        x = y = 0;
    }

    Point(ld xx, ld yy) {
        x = xx;
        y = yy;
    }

    void get() {
        cin >> x >> y;
    }

    bool operator ==(Point &a) {
        return x == a.x && y == a.y;
    }


    void operator -=(Point &a) {
        x -= a.x, y -= a.y;
    }

    void operator +=(Point &a) {
        x += a.x, y += a.y;
    }

    void operator *=(int p) {
        x *= p, y *= p;
    }

    Point operator -(Point &a) {
        Point c;
        c.x = x - a.x;
        c.y = y - a.y;
        return c;
    }

    Point operator +(Point &a) {
        Point c;
        c.x = x + a.x;
        c.y = y + a.y;
        return c;
    }

    void norm() {
        ld d = sqrt(x * x + y * y);
        x /= d, y /= d;
    }

    void print() {
        cout << "(" sp x << "," << y sp ")\n";
    }

};

ld cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}

ld dot(Point a, Point b) {
    return a.x * b.x + a.y * b.y;
}

ld dis(Point p, Point a) {
    return sqrt((p.x - a.x) * (p.x - a.x) + (p.y - a.y) * (p.
        y - a.y));
}
```

3.9 Polar Angle

```
// Returns polar angle in radians [-pi, pi]
ld polarAngle(Point p) {
    return atan2(p.y, p.x);
}

// Custom comparator to sort points around a pivot counter-
    clockwise
bool polarCompare(Point a, Point b, Point pivot) {
    Point da = a - pivot, db = b - pivot;
    ld crossVal = cross(da, db);
    if (abs(crossVal) < 1e-9) {
        return dis(pivot, a) < dis(pivot, b); // closer point
            first if collinear
    }
    return crossVal > 0; // counter-clockwise
}
```

3.10 Segment Intersection Check

```
bool onSegment(Point p, Point a, Point b) {
    return p.x >= min(a.x, b.x) && p.x <= max(a.x, b.x) &&
        p.y >= min(a.y, b.y) && p.y <= max(a.y, b.y);
}

bool intersectSegments(Point a, Point b, Point c, Point d) {
    ld o1 = orientation(a, b, c);
    ld o2 = orientation(a, b, d);
    ld o3 = orientation(c, d, a);
    ld o4 = orientation(c, d, b);

    if (((o1 > 0 && o2 < 0) || (o1 < 0 && o2 > 0)) &&
        ((o3 > 0 && o4 < 0) || (o3 < 0 && o4 > 0)))
        return true;

    if (abs(o1) < 1e-9 && onSegment(c, a, b)) return true;
    if (abs(o2) < 1e-9 && onSegment(d, a, b)) return true;
    if (abs(o3) < 1e-9 && onSegment(a, c, d)) return true;
    if (abs(o4) < 1e-9 && onSegment(b, c, d)) return true;
```

```
    return false;
}
```

4 Graph

4.1 Dijkstra

```
auto dik = [=](vector<vector<array<int, 2>>> &G, vector<int>
     &dp, int p) {
    priority_queue<array<int, 2>, vector<array<int,2>>,
         greater<>> pq;
    pq.push({0, p});

    while(pq.size()) {
        auto [c, v] = pq.top();
        pq.pop();

        if(c > dp[v]) continue;

        for(auto [uc, u] : G[v]) {
            if(dp[u] > c + uc) {
                dp[u] = c + uc;
                pq.push({dp[u], u});
            }
        }
    }
};

dik(G, dp, p);
```

5 Mathematics

5.1 Combinatorics

```
const int N = 1e6 + 9, mod = 1e9 + 7;

int f[N], inv[N], finv[N];
void prec()
{
    f[0] = 1;
    for (int i = 1; i < N; i++)
        f[i] = 1LL * i * f[i - 1] % mod;
    inv[1] = 1;
    for (int i = 2; i < N; i++)
    {
        inv[i] = (-(1LL * mod / i) * inv[mod % i]) % mod;
        inv[i] = (inv[i] + mod) % mod;
    }
    finv[0] = 1;
    for (int i = 1; i < N; i++)
        finv[i] = 1LL * inv[i] * finv[i - 1] % mod;
}

int npr(int n, int r)
{
    if (n < r || n < 0 || r < 0)
        return 0;
    return 1LL * f[n] * finv[n - r] % mod ;
}

int ncr(int n, int r)
{
    if (n < r || n < 0 || r < 0)
        return 0;
    return 1LL * f[n] * finv[n - r] % mod * finv[r] % mod;
}
```

5.2 Fast Exponentiation

```
int fexp(int a, int b) {
    int res = 1;
    while(b) {
        if(b & 1) res = (1LL * res * a) % MOD;
        a = (1ll * a * a) % MOD;
        b >>= 1;
    }
    return res;
}
```

5.3 Prime Generator

```
const int N = 1e6;
bitset<N + 1> is_prime;
vector<int> primes;

void sieve() {
    is_prime.set();
    is_prime[0] = is_prime[1] = 0;

    for (int i = 4; i <= N; i += 2)
        is_prime[i] = 0;

    for (int i = 3; 1LL * i * i <= N; i += 2) {
        if (is_prime[i]) {
            for (int j = i * i; j <= N; j += i << 1)
                is_prime[j] = 0;
        }
    }

    primes.push_back(2);
    for (int i = 3; i <= N; i += 2)
        if (is_prime[i])
            primes.push_back(i);
}
```

6 Misc

6.1 ahad

```
#include <bits/stdc++.h>

#ifdef AHAD
    #include "./debug.h"
    #define debug(...) debug_internal(__VA_ARGS__)
#else
    #define debug(...)
#endif

using namespace std;

using ll = long long;
using ld = long double;

#define sp << " " <<
#define pb push_back
#define F first
#define S second
#define PI acos(-1.0)

#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define sortall(x) sort(all(x))
#define sortrall(x) sort(rall(x))

#define printv(v) for(auto x : v) cout << x << " "; cout <<
     "\n"
#define printm(m) for(auto x : m) cout << x.F sp x.S << "\n"

#define make_unique(x) sortall(x); (x).resize(unique(all(x))
      - (x).begin())
```

```
#define numtobin(n, c) bitset<32>(n).to_string()
#define bintoint(bin_str) stoi(bin_str, nullptr, 2)

struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::
            steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

template<typename T1, typename T2>
using safe_map = unordered_map<T1, T2, custom_hash>;

template<typename T>
using safe_set = unordered_set<T, custom_hash>;

inline void fastIO() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
}

inline void fraction(int n) {
    cout.unsetf(ios::floatfield);
    cout.precision(n);
    cout.setf(ios::fixed,ios::floatfield);
}

inline void yn(bool res, bool cap = true) {
    vector<string> s = {"Yes", "YES", "No", "NO"};
    cout << ((res) ? s[0 + cap] : s[2 + cap]) << "\n";
}

inline ll nxt() { ll x; cin >> x; return x;}
inline ld nxtD() { ld x; cin >> x; return x;}
inline string nxtStr() { string x; cin >> x; return x;}

template<typename T, typename U>
void chmin(T &x, U y) {
    if (y < x) x = y;
}

template<typename T, typename U>
void chmax(T &x, U y) {
    if (y > x) x = y;
}

const int MOD = 1e9 + 7;
const int INF = 1e9;
const ll LINF = 1e18;

void proc(){

}

void solve() {


}

int main() {
    fastIO();
    proc();

    int tc = 1;
#ifdef test
    tc = nxt();
#endif
    for (int t = 1; t <= tc; t++) {
    #ifdef case
        cout << "Case " << t << ": ";
    #endif
        solve();
    }
}
```

6.2 Custom Comperator

```
// Custom Comperator
struct CustomCmp {
    bool operator()(const pair<int, int>& a, const pair<int,
         int>& b) const {
        if (a.first != b.first) {
            return a.first < b.first; // Ascending by first
                element
        }
        return a.second > b.second; // Descending by second
            element if first is equal
    }
};
```

7 Strings

7.1 KMP Algorithm

```
vector<int> createLps(string text) {
    vector<int> lps(text.length());
    int idx = 0;
    for(int i = 1; i < (int)text.length();) {
        if(text[idx] == text[i]) {
            lps[i] = idx + 1;
            i++, idx++;
        } else {
            if(idx != 0) idx = lps[idx-1];
            else lps[i] = idx, i++;
        }
    }

    return lps;
}

vector<int> kmp(string text, string pattern) {
    vector<int> ans, lps = createLps(pattern);
    int i = 0, j = 0;
    while(i < text.length()) {
        if(text[i] == pattern[j]) {
            i++, j++;
        } else {
            if(j != 0) j = lps[j - 1];
            else i++;
        }

        if(j == pattern.length()) {
            ans.pb(i - pattern.length());
            j = lps[j-1];
        }
    }

    return ans;
}
```